

PriorScope 1 차 MVP 설계서

LLMOps 교육 1 차 과제 설계서. 연구자가 투고하려는 논문과 기존 논문 1 편을 올리면, 에이전트가 내 논문의 핵심 기술 요소를 하나씩 기존 논문과 대응시키고, 근거 위치가 붙은 요소 대응표, 같은 실험 조건의 정량 비교, 차별성 리포트를 만든다. MVP 는 논문 1 편 x 기존 논문 1 편의 1:1 비교다. 기존 기획서(특허 비교)를 논문 대 논문 비교로 바꾸고, 1 차 가이드의 스타터 구조에 맞췄다.

1. 목적과 범위

논문은 기여(contribution)가 기존 연구와 무엇이 다른지 밝혀야 한다. 연구자가 어려워하는 것은 관련 논문을 찾는 일보다, 찾은 논문을 내 논문과 요소 단위로 대조하는 일이다. PriorScope 는 이 대조를 근거 위치와 함께 자동으로 해 준다.

항목	1 차 MVP 결정
주 사용자	논문 초안을 쓴 공학 계열 대학원생. 지도교수 미팅 전이나 리뷰에서 "기존 연구와 뭐가 다른가"를 지적받았을 때 쓴다
입력	PDF 2 개(투고할 내 논문, 비교할 기존 논문). 국문과 영문이 섞일 수 있다
대응 기준축	내 논문의 핵심 기술 요소(M1...Mn). 요소마다 기존 논문 요소(P1...Pn)에 이미 있는지 판정한다
강점 비교	정량 비교만 한다. 같은 데이터셋과 같은 지표의 결과가 양쪽에 있을 때만 수치를 나란히 비교한다
구현 도구	Upstage Studio 에이전트 3 개, Python(OpenAI 호환 SDK), Streamlit, Railway 배포

왜 에이전트인가: 요소 추출과 의미 대응은 Studio 에이전트가, 번호 검증과 커버리지·등급·수치 계산은 코드가 맡는다. 근거를 특정하지 못한 대응은 단정하지 않고 사람에게 넘긴다.

하지 않는 것: 비교할 논문 검색(2 차), 정성적 강점 서술(2 차), 여러 논문 동시 비교(2 차), 표절·심사 판정과 논문 우수성 평가, 스캔본 PDF 의 OCR 정확도 보장, 로그인과 사용자별 권한. 결과는 사전 점검용이다.

판정 원칙

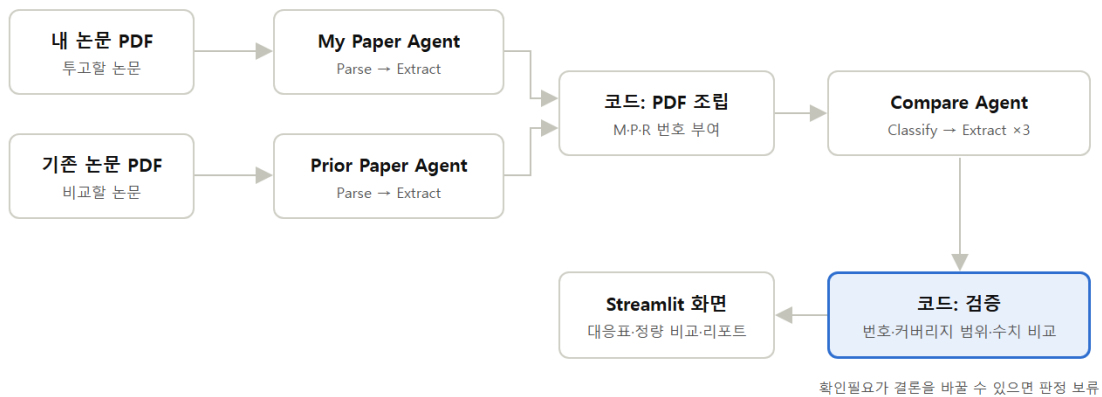
1. 허위 신규 주장 회피가 제 1 원칙이다. 근거를 특정하지 못한 대응은 "없음"이 아니라 "확인필요"로 보낸다. 검증 단계는 판정을 강등만 하고 승격하지 않는다.
2. 모든 대응 판정에는 근거 위치(섹션 번호)를 붙인다. 근거 없는 일치·유사·없음 판정은 확인필요로 내린다.
3. 숫자 계산(번호 검증, 커버리지, 등급, 수치 대소 비교)은 코드가 하고, 의미가 같은지의 판단만 Studio 에이전트가 한다.

2. 아키텍처

에이전트 3 개와 코드 층 2 곳으로 돌아간다. Studio 에이전트는 PDF 만 입력받으므로, 두 논문의 추출 결과(JSON)는 코드가 번호를 붙여 PDF 한 장으로 다시 조립해 Compare Agent 에 넘긴다.

실행 흐름

코드가 번호·범위·수치를 검증한 결과만 화면에 나간다



실행 흐름 · 에이전트 3 개 + 코드 층 2 곳

단계	입력	처리	출력
1	내 논문 PDF	My Paper Agent (Parse → Extract)	my_paper.json
2	기존 논문 PDF	Prior Paper Agent (Parse → Extract)	prior_paper.json
3	두 JSON	compare_builder.py (M-P-R 번호 부여, ReportLab)	Compare_Input.pdf

단계	입력	처리	출력
4	Compare_Input.pdf	Compare Agent (Parse → Classify → Extract ×3)	compare_result.json
5	세 JSON	validator.py (LLM 관여 없음)	verified_result.json

번호 부여: core_techniques 배열 순서대로 내 논문은 M1...Mn, 기존 논문은 P1...Pn, experiment_results 순서대로 R-M1..., R-P1...을 붙인다. builder 와 validator 가 같은 함수(ids.py 의 assign_ids)를 쓰므로, 같은 JSON 에서는 항상 같은 번호가 나온다.

실행 디렉터리: 실행마다 runs/<run_id>/ 폴더를 만들어 결과 파일 5 개를 저장한다. 공개 URL 에서 동시에 실행해도 결과가 섞이지 않는다. run_id 는 YYYYMMDD-HHMMSS-<6 자리 랜덤> 형식이며, runs/는 Git 에서 제외한다.

파일 구조

스타터 hr-ai-service-workflow/는 참고용으로 그대로 두고, llmops/priorscope/에 복사해 수정한다.

파일	상태	역할
agent_client.py	수정(소)	Studio 호출. 상태 확인에 최대 대기 시간(기본 180 초) 추가
upload.py, file_manager.py	그대로	Files API 업로드, JSON 입출력
config.py	수정	에이전트 ID·CONFIG ID 6 개, 경로를 run_paths(run_dir) 함수로 제공
my_paper_agent.py, prior_paper_agent.py, compare_agent.py	이름 변경	스타터의 resume / jd / matching 에이전트 모듈
ids.py	신규	assign_ids(): builder 와 validator 가 공유하는 번호 부여
compare_builder.py	신규	matching_builder.py 대체. 번호가 붙은 비교용 PDF 생성(나눔고딕)
validator.py	신규	코드 검증 층. 순수 함수로만 구성

파일	상태	역할
app.py	수정	1~5 단계 실행. --from-json <run_dir>로 비교 단계부터 재실행
service.py	수정	run_compare_service(my_path, prior_path, on_step). 실패하면 즉시 중단
streamlit_app.py	신규 작성	결과 화면
agent_configs/*.json	교체	Studio 에서 내보낸 스키마 정답지 3 개
tests/, eval/	신규	단위 테스트(Studio 호출 없음), 평가 라벨과 evaluate.py

스타터에서 고치는 동작: service.py 는 단계가 실패해도 계속 진행하고, 마지막에 디스크에 남은 이전 결과를 읽어 보여준다. 다른 논문의 예전 결과가 지금 결과처럼 표시될 수 있으므로 이 동작을 없애고 실패하면 StepError 를 던진다. agent_client.py 의 무한 상태 확인 루프에는 최대 대기 시간을 넣는다.

3. 에이전트 스키마

Studio Extract 결과는 모든 필드가 배열이다. "단일"로 적은 필드는 원소를 1 개만 두고 코드가 첫 값을 읽는다. 두 Paper Agent 는 분석 대상이 달라 따로 두되, 공통 필드는 이름·유형·목록 여부를 똑같이 맞춘다.

공통 요소 분해 규칙: 두 Paper Agent 의 core_techniques 설명에 같은 문구를 넣는다. "원소 하나에는 기법·모듈·학습 절차를 하나만 담는다. 쉽표나 세미콜론으로 여러 개를 합치지 않는다. 형식은 [섹션번호] 원문 표현 | 한국어 설명이다. 원문 표현은 논문의 원래 언어를 그대로 쓴다." 한국어 설명을 함께 뽑아 국문·영문이 섞여도 대응할 수 있게 한다.

My Paper Agent / Prior Paper Agent 스키마 (Parse → Extract)

항목명	설명	My	Prior
title	단일. 논문 제목	✓	✓
language	단일. ko 또는 en	✓	✓

항목명	설명	My	Prior
research_field	단일. 연구 분야	✓	✓
problem	해결하려는 문제 또는 기존 방법의 한계	✓	✓
core_techniques	공통 분해 규칙을 따르는 핵심 기술 요소. 대응표의 단위	✓	✓
experiment_results	제안한 방법의 결과만. 데이터셋 실험 설정 지표 수치 ↑ 또는 ↓ [표/섹션]	✓	✓
keywords	핵심 키워드 3~6 개	✓	✓
contributions	저자가 주장하는 기여점. [섹션] 내용	✓	—
citation_info	단일. 저자, 학회 또는 저널, 연도	—	✓
limitations	기존 논문이 스스로 밝힌 한계. [섹션] 내용	—	✓

Compare Agent 분류 기준 (Classify)

분류	기준
High Overlap	내 논문의 핵심 요소 대부분이 기존 논문에 이미 있어서 차별성 보강이 꼭 필요한 경우
Medium Overlap	일부 요소가 겹치지만 내 논문에만 있는 구성이 있는 경우
Low Overlap	분야나 개념만 비슷하고 핵심 구성은 다른 경우

Compare Agent 결과 스키마 (Extract 3 개 모두 동일)

항목명	설명
overlap_level	단일. Classify 결과
llm_score	단일, 0~100. 참고값으로만 쓴다
summary	단일. 두 논문의 관계 2~3 문장

항목명	설명
element_mapping	M 요소마다 한 원소. "M2 ↔ P4,P5 유사 M[3.1] / P[2.3] 설명". 대응이 없으면 "M3 ↔ - 없음 검토 범위 설명". 판정은 일치·유사·없음·확인필요 중 하나
result_pairs	"R-M1 ~ R-P2 동일 조건 또는 조건 다름 설명". 데이터셋과 지표가 같은 쌍만. 실험 설정까지 같으면 동일 조건, 다르면 조건 다름
caution_points	차별성을 분명히 해야 할 지점
differentiation_suggestions	바로 쓸 수 있는 서술 제안 3 개
related_work_draft	단일. 내 논문 언어로 쓴 1~2 문장. citation_info 를 인용

Extract 지시문에는 "같은 기술을 다른 용어나 다른 언어로 쓴 경우도 대응으로 본다", "확신하지 못하면 없음이 아니라 확인필요로 적는다"를 넣는다. 겹치는 요소와 고유 요소는 따로 묻지 않고 코드가 element_mapping 에서 파생하므로, 대응표와 요약이 서로 어긋날 수 없다.

비교용 PDF 구성: 안내문(번호 체계와 출력 형식) → 내 논문(제목, 문제, M1...Mn, 기여점, R-M) → 기존 논문(제목, 인용 정보, 문제, P1...Pn, 한계, R-P) 순서로 조립한다.

추출 방법

추출은 코드가 아니라 Studio 에이전트 안의 두 노드가 한다. Parse 노드(Upstage Document Parse)는 PDF 를 제목·문단·표·목록 구조를 살린 텍스트로 바꾼다. 표의 행과 열이 유지되므로 실험 결과 표를 읽을 수 있고, 스캔본은 OCR 로 처리한다. Extract 노드(Information Extract)는 필드마다 LLM 이 문서를 읽고 값을 채운다. 필드의 설명이 사실상 프롬프트이므로, 추출 품질은 설명 문구로 정해진다. 샘플 논문으로 Studio 화면에서 실행해 보며 문구를 다듬는다.

필드	Extract 설명(요지)	주로 읽는 곳
title	논문 제목 하나	첫 페이지 상단
language	본문 언어. ko 또는 en	본문
research_field	연구 분야 한 구절	초록, 키워드

필드	Extract 설명(요지)	주로 읽는 곳
problem	해결하려는 문제나 기존 방법의 한계. 문제 하나당 원소 하나	초록, 서론
core_techniques	공통 분해 규칙	방법론 섹션 소제목과 본문
experiment_results	아래 형식. 제안 방법의 결과만	실험 섹션의 표
keywords	키워드 3~6 개	키워드 란, 초록
contributions	저자가 명시한 기여("our contributions are...")	서론 끝부분
citation_info	저자, 학회 또는 저널, 연도	첫 페이지 헤더·푸터
limitations	저자가 스스로 밝힌 한계	결론, Limitations 섹션

LLM 추출이라 정확도가 100%는 아니다. 코드의 형식 검사(4 장)와 평가 샘플(7 장)로 보완한다.

experiment_results 형식 (6 칸)

실험 결과 한 건을 구분자 | 로 이은 문자열 하나로 받는다. 필드별 배열로 따로 받으면 원소 하나만 빠져도 짝이 어긋나기 때문이다. 예: ImageNet-1k | ViT-B/16, 300ep | Top-1 Acc (%) | 83.1 | ↑ | Table 2

칸	규칙
데이터셋	논문에 적힌 이름 그대로
실험 설정	수치를 얻은 조건(모델 크기, 백본, 학습량, 추가 데이터, 평가 방식 등). 없으면 -
지표	단위 포함
수치	표에 적힌 숫자. ± 표준편차는 있어도 됨
방향	높을수록 좋으면 ↑, 낮을수록 좋으면 ↓. 논문 표시가 없으면 지표 성격으로 정함(정확도 ↑, 오류율·지연 시간 ↓)

칸	규칙
위치	근거 표 또는 섹션

실험 설정을 두는 이유: 데이터셋과 지표가 같아도 조건이 다르면(예: BERT-base vs BERT-large) 공정한 비교가 아니다. Compare Agent 가 실험 설정을 보고 "조건 다름"으로 표시하면 코드는 대소 비교를 하지 않는다. 제안한 방법의 결과만 담고, (데이터셋, 실험 설정, 지표) 조합 하나당 한 줄로 적는다.

4. 코드 검증 총

validator.py 는 my_paper.json, prior_paper.json, compare_result.json 을 읽어 verified_result.json 을 만든다. 언어 모델은 관여하지 않으며, 의심스러우면 확인필요로 내린다.

입력 점검: 어느 한쪽이라도 core_techniques 가 비어 있으면 "논문으로 인식되지 않았습니다"와 해당 파일을 안내하고 중단한다. 두 파일이 같은 파일(SHA-256 동일)이면 분석 전에 중단한다.

element_mapping 파싱과 판정 강등

상황	처리
형식이 깨진 줄	버리고 dropped 에 "형식 오류"로 기록
존재하지 않는 M 번호	그 줄 제거
존재하지 않는 P 번호	그 P 만 제거. 남은 P 가 없는데 일치·유사면 확인필요
대응 줄이 없는 M	확인필요 (대응 판정 누락)
같은 M 에 서로 다른 판정	확인필요 (판정 충돌)
일치·유사인데 M[섹션] 또는 P[섹션] 근거가 없음	확인필요
인용한 섹션이 실제 요소의 섹션 태그와 다름	확인필요
없음인데 검토 범위가 비어 있음	확인필요
판정 값이 4 중 밖	확인필요

커버리지와 등급 (범위 규칙)

확인필요 요소는 실제로 일치일 수도, 없음일 수도 있다. 그래서 커버리지를 한 값이 아니라 범위로 계산하고, 확인필요가 결론을 뒤집을 수 있으면 등급을 확정하지 않는다. 이 규칙은 결과 화면 맨 위의 최종 등급 배지에만 적용되며, 요소별 판정에는 영향이 없다.

1. 최소 커버리지 = $(\text{일치 수} + \text{유사 수} \times 0.5) \div M \text{ 요소 수} \times 100$. 확인필요를 모두 "없음"으로 가정한 값이다.
2. 최대 커버리지 = $(\text{일치 수} + \text{확인필요 수} + \text{유사 수} \times 0.5) \div M \text{ 요소 수} \times 100$. 확인필요를 모두 "일치"로 가정한 값이다.
3. 등급 구간은 70 이상 High, 30 이상 70 미만 Medium, 30 미만 Low 다. 최소와 최대가 같은 구간이면 확정, 다르면 판정 보류로 범위(예: Low~Medium)를 표시한다.
4. 교차 검증은 확정일 때만 Classify 결과와 비교해 일치 또는 확인필요로 표시한다. 보류면 Classify 등급은 참고값으로만 쓴다.

일치	유사	없음	확인필요	최소 ~ 최대	결과
1	0	8	1	10% ~ 20%	Low 확정. 확인필요가 겹쳐도 Low
1	0	6	3	10% ~ 40%	판정 보류 (Low~Medium). "새롭다"고 단정하면 위험
8	0	0	2	80% ~ 100%	High 확정

예시는 내 논문 요소가 10 개인 경우다. 확인필요 3 개 때문에 Low 가 Medium 으로 바뀔 수 있는데도 Low 로 표시하면, 사용자는 "기존 연구와 거의 안 겹친다"고 잘못 보고할 수 있다. 범위 규칙은 이런 허위 신규 주장을 코드로 막는다.

파생 목록: 일치·유사 판정은 겹치는 요소, 검증을 통과한 없음은 내 논문 고유 요소(차별점), 확인필요는 사유와 함께 확인 필요 목록으로 분류한다.

정량 비교 (result_pairs)

1. R-M, R-P 번호가 실제로 있는지 확인한다. 없으면 dropped 에 기록한다.
2. 6 칸으로 쪼갠다. 칸 수가 다르면 dropped 에 "형식 오류"로 기록한다.
3. 수치에서 %, ± 이하, 십표, 공백을 지우고 숫자로 읽는다.

4. Compare Agent 가 "조건 다름"으로 표시했거나, 숫자로 읽을 수 없거나, 한쪽 방향(↑/↓)이 없거나, 양쪽 방향이 다르거나, 두 값의 비가 50 배 이상(단위가 다를 가능성)이면 "비교 불가"와 사유를 남긴다.
5. 그 밖의 경우 방향에 따라 우세·열세·동일을 판정하고, 절대 차이(내 수치 - 기존 수치, 지표 단위 그대로)와 상대 차이((내 수치 - 기존 수치) ÷ |기존 수치| × 100, 소수점 첫째 자리)를 남긴다. 기존 수치가 0 이면 상대 차이는 비운다.
6. 화면에는 "+1.3%p (상대 +1.6%)"처럼 절대 차이를 앞에, 상대 차이를 괄호에 둔다. %인 지표는 절대 차이에 %p 를 붙여 상대 차이(%)와 헷갈리지 않게 한다.

내 논문	기존 논문	판정
ImageNet-1k ViT-B/16 Top-1 Acc (%) 83.1 ↑	ImageNet-1K ViT-B/16 Top-1 정확도 81.8 ↑ (동일 조건)	우세, +1.3%p (상대 +1.6%)
COCO R-50 Latency (ms) 2.0 ↓	COCO R-50 Latency (ms) 4.0 ↓ (동일 조건)	우세, -2.0 ms (상대 -50.0%)
... Acc 0.831 ↑	... Acc (%) 81.8 ↑	비교 불가 (비 약 98 배, 단위 의심)
GLUE BERT-base 평균 84.0 ↑	GLUE BERT-large 평균 86.0 ↑ (조건 다름)	비교 불가 (조건 다름)

통계적 유의성 검정, 여러 지표를 합친 종합 점수, 표준편차를 고려한 판정은 하지 않는다.

화면에는 "수치상 우세/열세"로만 표시한다.

verified_result.json 주요 항목

항목명	설명
level, level_status	등급(Low / Medium / High 또는 Low~Medium 같은 범위), 확정 또는 보류
coverage_min, coverage_max	최소·최대 커버리지(%)
classify_level, cross_check	Classify 등급, 교차 검증 결과(일치 / 확인필요 / 참고)
counts	판정별 개수(일치·유사·없음·확인필요)

항목명	설명
mappings	M 마다 내 요소, 대응 P 요소, 판정, 근거, 설명, 강등 사유
overlapping, novel, uncertain	겹치는 요소, 고유 요소, 확인 필요 목록(사유 포함)
result_comparisons	데이터셋, 실험 설정(내/기준), 조건(동일/다름), 지표, 내 수치, 기존 수치, 방향, 결과, 절대 차이, 상대 차이, 사유
report	summary, caution_points, differentiation_suggestions, related_work_draft
dropped	제거된 결과 사유(디버깅용)

5. 화면 항목

화면은 Streamlit 한 페이지이며, Railway 공개 URL 로 배포한다. 위에서 아래로 다음 순서를 따른다.

1. 업로드 2 개(투고할 내 논문 PDF / 비교할 기존 논문 PDF), [비교 시작] 버튼, 샘플 PDF 다운로드. 파일이 빠지면 어느 쪽인지 안내한다.
2. 진행 상태: 1~5 단계를 표시하고, 실패하면 몇 단계에서 왜 실패했는지 보여주고 멈춘다.
3. 결과 배지: 등급(확정 또는 보류와 범위), 커버리지(값 또는 범위), 교차 검증, 판정 개수.
4. 요소 대응표: M 마다 내 요소(원문 + 한국어 설명), 대응 P 요소, 판정(색 구분), 근거 위치, 강등 사유.
5. 차별점과 겹침: 내 논문 고유 요소와 기존 논문과 겹치는 요소를 2 열로 보여준다.
6. 정량 비교 표: 데이터셋, 실험 설정(내/기준), 지표, 내 수치, 기존 수치, 결과, 차이(절대·상대)와 사유. 비교할 쌍이 없으면 "직접 비교 가능한 실험이 없습니다"를 표시한다.
7. 차별성 리포트: 요약, 주의 지점, 서술 제안 3 개.
8. Related Work 초안과 복사 버튼.
9. 확인 필요 목록: 접지 않고 별도 섹션으로 보여준다. 보류면 "이 항목을 확인하면 등급이 확정됩니다"를 안내한다.
10. verified_result.json 과 compare_result.json 다운로드, 하단 고지("사전 점검용이며 심사·표절 판정을 대체하지 않습니다").

6. 오류 처리

실패하면 멈추고, 어디서 왜 실패했는지 보여준다. 이전 결과로 대신 채우지 않으며 자동 재시도는 하지 않는다.

상황	처리
업로드 실패, Job 생성 실패, status=failed	단계와 메시지를 담은 <code>StepError</code> 를 화면에 표시하고 중단
최대 대기 시간(180 초) 초과	타임아웃 오류를 <code>StepError</code> 로 표시
output_text 가 없거나 JSON 이 아님	해당 단계 오류로 처리
스키마 필드 누락	빈 배열로 보고 <code>dropped</code> 에 기록. <code>core_techniques</code> 가 비면 입력 점검에서 중단
같은 파일 두 번 업로드	분석 전에 안내하고 중단
논문이 아닌데 그럴듯한 요소가 추출되는 문서	막지 못한다. README 에 한계로 적는다
스캔본 PDF	Parse OCR 에 맡기되 정확도는 보장 범위에서 뺀다

7. 테스트와 평가 계획

단위 테스트: pytest 로 Studio 호출 없이 손으로 만든 JSON 픽스처를 써서, 번호 부여의 결정성, 판정 강등 규칙, 범위 규칙과 교차 검증, 정량 비교의 모든 분기, 비교용 PDF 생성을 하나씩 확인한다.

통합 확인: 샘플 쌍으로 python app.py 를 실행해 1~5 단계가 오류 없이 끝나고 결과 파일 5 개가 생기는지 확인한다.

평가는 정답을 미리 아는 샘플 5 쌍으로 하며, 주 지표는 허위 신규 주장률이다. 구체적인 논문은 팀에서 고른다.

샘플	구성	기대 결과
S1	같은 연구의 학회판 ↔ 저널 확장판	High
S2	후속 논문 ↔ 원 논문 (예: RoBERTa ↔ BERT)	Medium

샘플	구성	기대 결과
S3	분야가 다른 논문 쌍	Low
S4	국문 논문 ↔ 같은 기술의 영문 논문	언어가 섞여도 대응 유지
S5	논문이 아닌 문서	판정 없이 입력 확인 안내

정답 고정: M 번호는 추출 결과에 따라 달라지므로, 샘플마다 my_paper.json 과 prior_paper.json 을 한 번 만들어 eval/samples/S*/에 고정한다. 팀원 2 명이 고정본의 M 요소별 정답을 labels.json 에 라벨링하고, 의견이 갈린 요소는 정답을 확인필요로 둔다.

eval/evaluate.py: compare 모드는 고정 JSON 으로 비교 단계만 실행해 요소 단위 지표를 계산한다. full 모드는 원본 PDF 로 전체를 3 회 실행해 등급 일관성과 처리 시간을 잰다. 결과는 eval/report.json 과 콘솔 표로 낸다.

지표	목표	성격
허위 신규 주장률 (정답이 일치·유사인 M 중 "없음"으로 판정된 비율)	10% 이하	주 지표
요소 대응 정확도 (판정이 정답과 같은 M 비율)	80% 이상	품질 지표
등급 일치율 (확정 등급 기준, 보류는 별도 집계)	80% 이상	Classify·범위 규칙
근거 인용 유효율 (인용 번호와 섹션이 실제로 있음)	100%	코드 검증
정량 비교 정확도 (대소 판정이 수동 계산과 같음)	100%	코드 계산
3 회 반복 시 등급 일관성	3 회 모두 같음	재현성
처리 시간	측정만 함	비기능

8. 배포, 리스크와 2 차 확장

배포: GitHub 저장소를 Railway 에 연결하고, UPSTAGE_API_KEY 와 에이전트 ID·CONFIG ID 6 개, PORT 를 환경변수로 등록한다. 시작 명령은 streamlit run streamlit_app.py --server.port=\$PORT --server.headless=true 이다. .env 와 runs/는 Git 에서 제외한다.

리스크	영향	대응
요소 분해가 실행마다 다르다	커버리지가 흔들림	공통 분해 규칙, 평가 시 추출 결과 고정, 등급 일관성 측정
국문·영문 용어 차이	대응 누락	한국어 설명 필드, s4 로 확인, 불확실하면 확인필요
Extract 3 개 스키마 불일치	화면 코드 KeyError	스키마 하나를 복사해 쓰고 agent_configs 로 대조, 누락 필드는 빈 배열로 처리
언어 모델이 "신규"를 단정	허위 신규 주장	근거 없는 판정 강등, 범위 규칙으로 등급 보류
긴 논문으로 처리 지연	실행 시간 초과	최대 대기 시간 오류 표시, 처리 시간 측정
Extract 가 여러 요소를 한 문자열로 합침	M 개수 왜곡	분해 규칙 명시, 평가에서 확인

2 차 확장: Studio 대신 Python 에서 LLM 을 직접 호출하는 LangGraph 구조로 옮기고, FastAPI-Docker 로 운영한다. ChromaDB 로 내 논문과 비슷한 기존 논문을 자동으로 찾아 1 차에서 하지 않는 검색 단계를 보완하고, 여러 논문 동시 비교와 정성적 강점 서술을 추가한다. 1 차의 validator.py 와 eval/은 그대로 재사용한다.